

UCS503 – Software Engineering Lab

Campus Resource Management System

Campus Reserve

Software Engineering Prototype Report

Submitted by

Aditya Bhardwaj 1024030777

Ayati Kalra 1024030783

Rishita Jain 1024030782

Group: _____

Submitted to

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Repository-based prototype documentation

September 2026

Contents

Abstract	iii
1 Project Selection Phase	1
1.1 Software Bid	1
1.2 Project Overview	1
1.3 Objectives and Boundaries	1
1.4 Proposed Completion Plan	2
2 Analysis Phase	3
2.1 Use Cases	3
2.1.1 Use-Case Diagram	3
2.1.2 Use-Case Specifications	4
2.1.3 UC-01: Browse Availability	4
2.1.4 UC-02: Submit Reservation	4
2.1.5 UC-03: Track or Cancel Booking	4
2.1.6 UC-04: Review Request	5
2.1.7 UC-05: Manage Rooms and Societies	5
2.1.8 UC-06: View Notifications	5
2.2 Business Rules	5
2.3 Activity and Swimlane Diagrams	6
2.3.1 Activity Diagram: Reservation Submission	6
2.3.2 Swimlane Diagram: Request to Decision	6
2.4 Sequence and Collaboration Diagrams	7
2.4.1 Sequence Diagram: Submit a Booking	7
2.4.2 Collaboration Diagram: Review a Request	7
2.5 Conceptual Class and Data Model	8
2.5.1 Reservation Status Model	8
2.6 Data Flow Diagrams	9
2.6.1 DFD Level 0: System Context	9
2.6.2 DFD Level 1: Core Processes	9
2.6.3 DFD Level 2: Reservation Validation	10
2.6.4 Conflict Rule and Examples	10
2.7 Software Requirements Specification	11

2.7.1	Introduction and Overall Description	11
2.7.2	Functional Requirements	11
2.7.3	Non-functional Requirements and Constraints	12
2.7.4	External Interfaces	12
2.7.5	Acceptance Boundary	12
2.8	User Stories and Story Cards	13
3	Prototype Design and Technology	14
3.1	Technology Stack	14
3.2	System Architecture	14
3.3	Prototype Screens and Navigation	15
3.4	Eight-step Booking Prototype	15
3.5	Illustrative Demonstration Scenario	15
4	Validation and Future Scope	16
4.1	Acceptance Test Plan	16
4.2	Evidence and Unverified Areas	16
4.3	Limitations and Future Scope	17
4.4	Conclusion	17
	References	18

Abstract

Campus Reserve is a campus resource booking prototype for student societies, permission in-charges and campus administrators. It brings room availability, reservation requests, review decisions and booking records into one browser interface. Students select a campus space and time, complete an eight-step request form, and track its status. Permission in-charges approve or reject requests, while administrators maintain room and society records.

The current implementation uses React, TypeScript and Vite, with browser localStorage for persistence. It demonstrates workflow and interface behaviour without a backend, shared database or production authentication. This report adapts the supplied UCS503 report's project-selection and analysis structure to the linked repository. It includes use-case specifications, activity and interaction models, a conceptual data model, data-flow diagrams, requirements, user stories and a prototype demonstration plan.

Scope of evidence. Descriptions derive from repository documentation and inspected source files on 15 September 2026. Diagram models describe or abstract the current implementation unless marked as proposed. No deployment, load benchmark or independent application test result is claimed. The reference report provides structure only; its health-chatbot content and contributor details do not apply to this project.

Chapter 1

Project Selection Phase

1.1 Software Bid

Campus activities require suitable spaces and a clear permission process. A booking workflow spread across informal messages and separate records can make availability and approval status difficult to follow. This is the problem the project addresses; no survey or measured baseline is claimed.

The proposed campus portal provides a common interface for discovering rooms, submitting society requests and recording review decisions. Its prototype supports an academic demonstration of software analysis, interface design, validation and state management. The present scope focuses on rooms and campus spaces. Equipment requirements can be recorded within an activity request, but the repository does not implement a separate equipment inventory or lending system.

1.2 Project Overview

- Students and society representatives browse availability and submit reservation requests.
- Permission in-charges review pending requests and record approval or rejection.
- Administrators add or update room and society information.
- Local notifications communicate booking submissions and status changes.
- The browser preserves demo data through localStorage.

1.3 Objectives and Boundaries

Prototype objective	Current boundary
Make availability visible	Data is local to one browser, not synchronized campus-wide.
Prevent overlapping requests	Client-side checks compare Pending and Approved bookings.
Demonstrate role-specific workflows	Demo persona switching does not verify passwords.
Maintain booking history	Local records can be cleared or changed by the browser user.
Capture permission decisions	No authoritative institutional approval integration exists.

1.4 Proposed Completion Plan

Stage	Work	Exit condition
1	Review requirements and prototype	Faculty accepts scope and workflows.
2	Demonstrate booking and review paths	Core scenarios produce expected state changes.
3	Improve validation and access controls	Agreed acceptance criteria pass.
4	Introduce API and database	Shared persistence and server-side conflict control work.
5	Pilot and document	User feedback and test evidence support deployment.

This is a proposed next-stage plan, not a claim about completed milestones or a fixed semester schedule.

Chapter 2

Analysis Phase

2.1 Use Cases

2.1.1 Use-Case Diagram

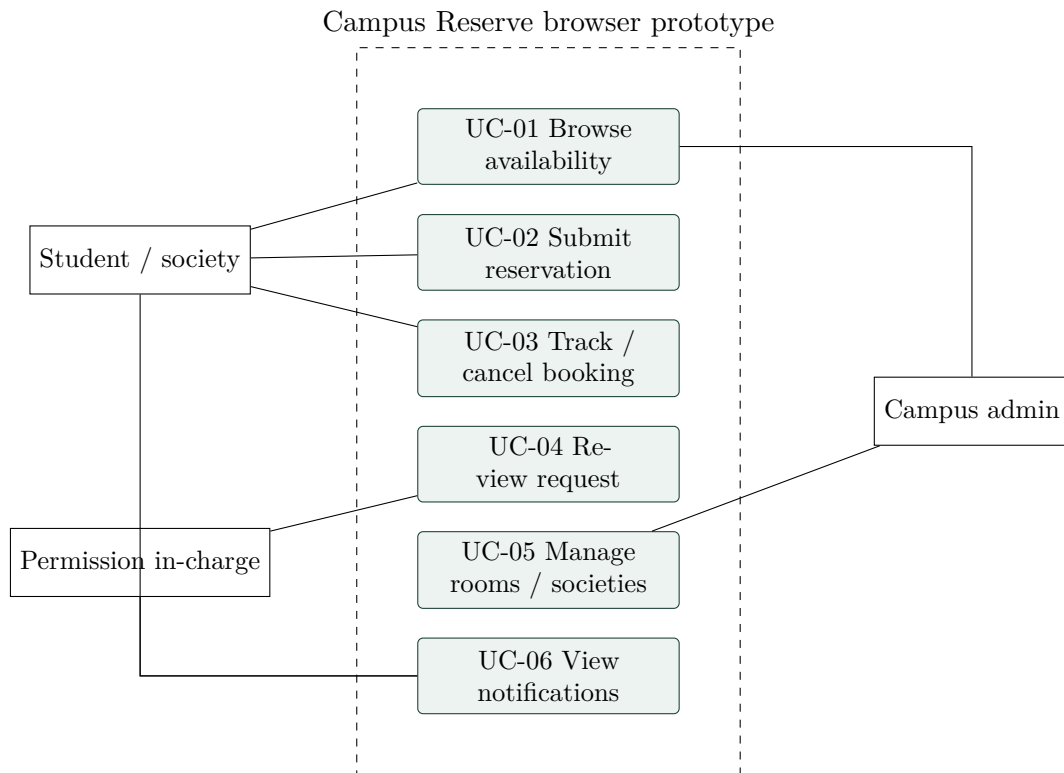


Figure 2.1: Primary actor interactions. Shared availability is available across roles.

Supporting functions: form validation, overlap detection, local persistence and notification creation support these user goals. They are internal functions rather than separate human actors. Persona selection simulates role access for the demonstration.

2.1.2 Use-Case Specifications

2.1.3 UC-01: Browse Availability

Actor	Student, permission in-charge or administrator
Preconditions	The prototype has initialized room and booking data.
Main flow	Select location, room and date; inspect the hourly schedule; select an available slot to open the booking form with the room and time populated.
Alternate flow	Choose a different date, time or room if the displayed slot is blocked or unsuitable.
Postcondition	The actor sees availability, or a student proceeds with a selected slot.

2.1.4 UC-02: Submit Reservation

Actor	Student / society representative
Preconditions	A demo persona is active and a suitable room and society record exist.
Main flow	Choose activity, society, location and room; enter date and time; choose permission type; complete activity details; review and submit.
Alternate flow	Invalid dates, reversed times, unavailable rooms, conflicts or missing required details require correction. An end time after 20:00 requires Night Permission.
Postcondition	The prototype stores a Pending booking with an identifier and creates an in-charge notification.

2.1.5 UC-03: Track or Cancel Booking

Actor	Student / society representative
Preconditions	The browser contains booking records visible in the student's booking screen.
Main flow	Open My Bookings; search or filter by status; inspect details; use the available cancellation action and confirm.
Alternate flow	Cancel the confirmation dialog to leave the record unchanged. Records already in terminal states may not offer cancellation in the interface.
Postcondition	The actor sees the current status; a confirmed cancellation records Cancelled and creates a notification.

2.1.6 UC-04: Review Request

Actor	Permission in-charge
Preconditions	A Pending request exists in the local browser data.
Main flow	Open the pending queue; inspect activity and timing details; approve with optional remarks or reject with a reason; review the updated status.
Alternate flow	Close the dialog without deciding, or correct missing decision information requested by the interface.
Postcondition	The booking becomes Approved or Rejected, review metadata is stored, and the prototype creates a student notification.

2.1.7 UC-05: Manage Rooms and Societies

Actor	Campus administrator
Preconditions	An admin demo persona is active.
Main flow	Open Rooms or Societies; add or edit the selected record; save the changes; inspect the updated list.
Alternate flow	Correct invalid or incomplete form entries, or dismiss the form without saving.
Postcondition	The local room or society collection reflects the saved changes.

2.1.8 UC-06: View Notifications

Actor	Student or permission in-charge
Preconditions	Local notifications exist for booking activity.
Main flow	Open the notification panel or page; inspect messages; mark a notification or all notifications as read.
Alternate flow	An empty list or a list of already-read messages requires no action.
Postcondition	Read-state changes persist in the browser. Notification routing uses recipient roles rather than verified individual recipients.

2.2 Business Rules

1. A reservation contains a society, campus room, date, start time, end time, activity and permission type.
2. New reservations have status Pending. Approval, rejection and cancellation update that stored status.
3. Pending and Approved reservations block overlapping intervals for the same room and date. Rejected and Cancelled reservations do not.
4. Adjacent intervals are allowed. A booking ending at 11:00 does not conflict with a booking starting at 11:00.
5. End time must follow start time; the form rejects past dates and rooms marked Unavailable.
6. In the current form, an end time later than 20:00 requires Night Permission.

2.3 Activity and Swimlane Diagrams

2.3.1 Activity Diagram: Reservation Submission

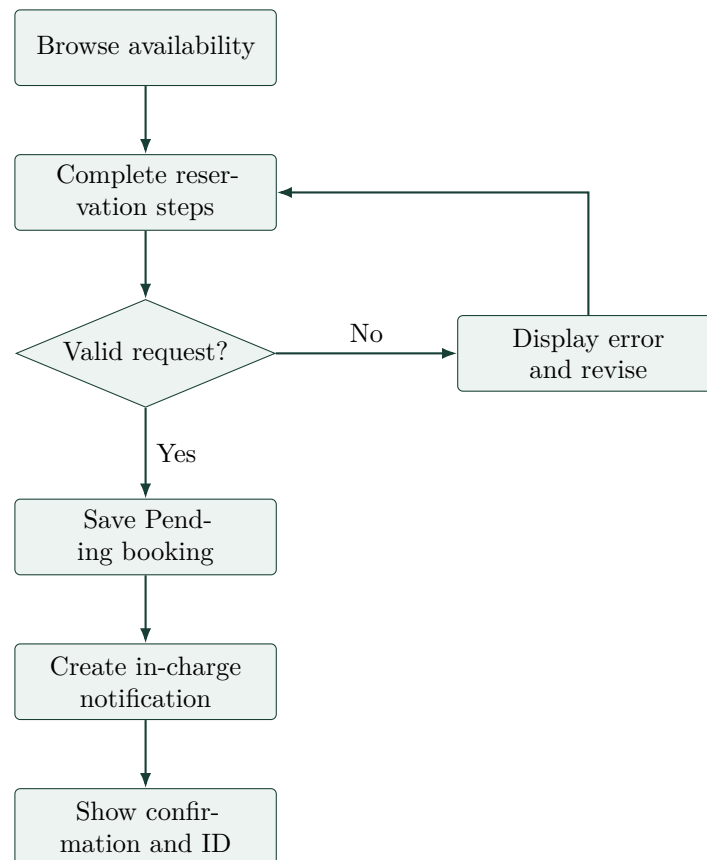


Figure 2.2: Form checks precede a local booking write.

2.3.2 Swimlane Diagram: Request to Decision

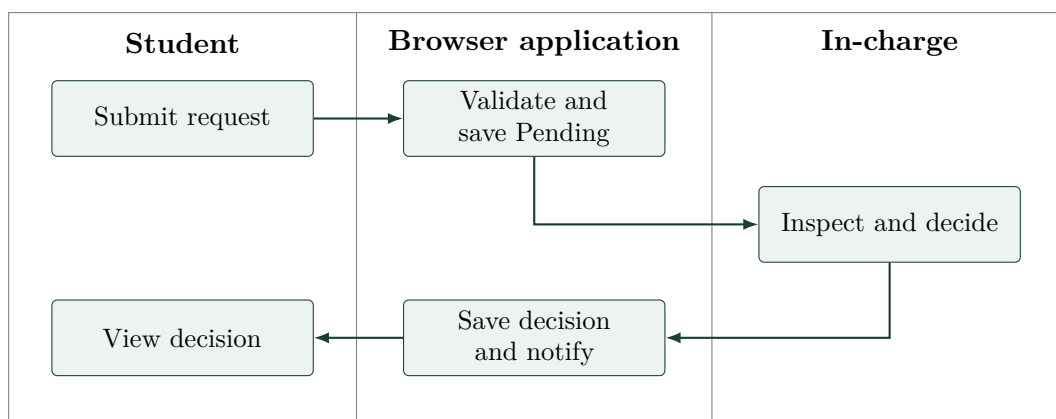


Figure 2.3: Role responsibilities. Persona switching occurs within the same browser demo.

2.4 Sequence and Collaboration Diagrams

2.4.1 Sequence Diagram: Submit a Booking

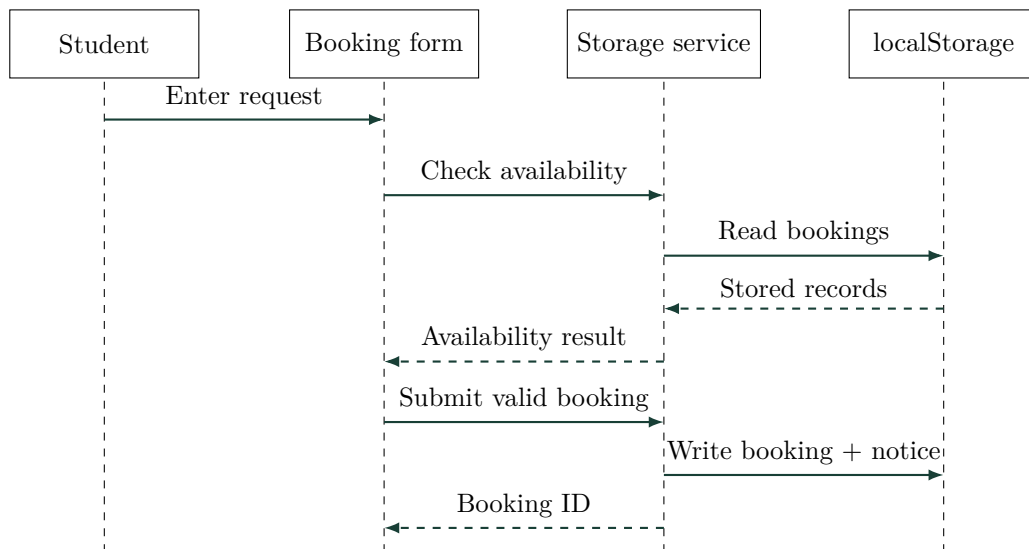


Figure 2.4: Successful submission path. Invalid requests stay in the form for correction.

2.4.2 Collaboration Diagram: Review a Request

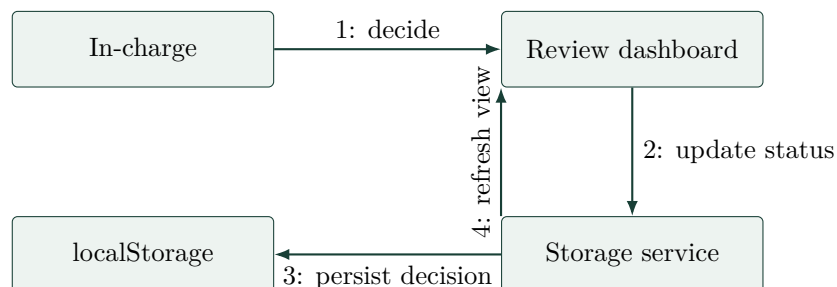


Figure 2.5: Numbered collaboration messages. The storage service also creates the status notification.

2.5 Conceptual Class and Data Model

The following model abstracts TypeScript interfaces in `src/types/index.ts`. These are data records rather than implemented object-oriented classes or database tables. Identifiers connect records in browser storage, without database-enforced foreign keys.

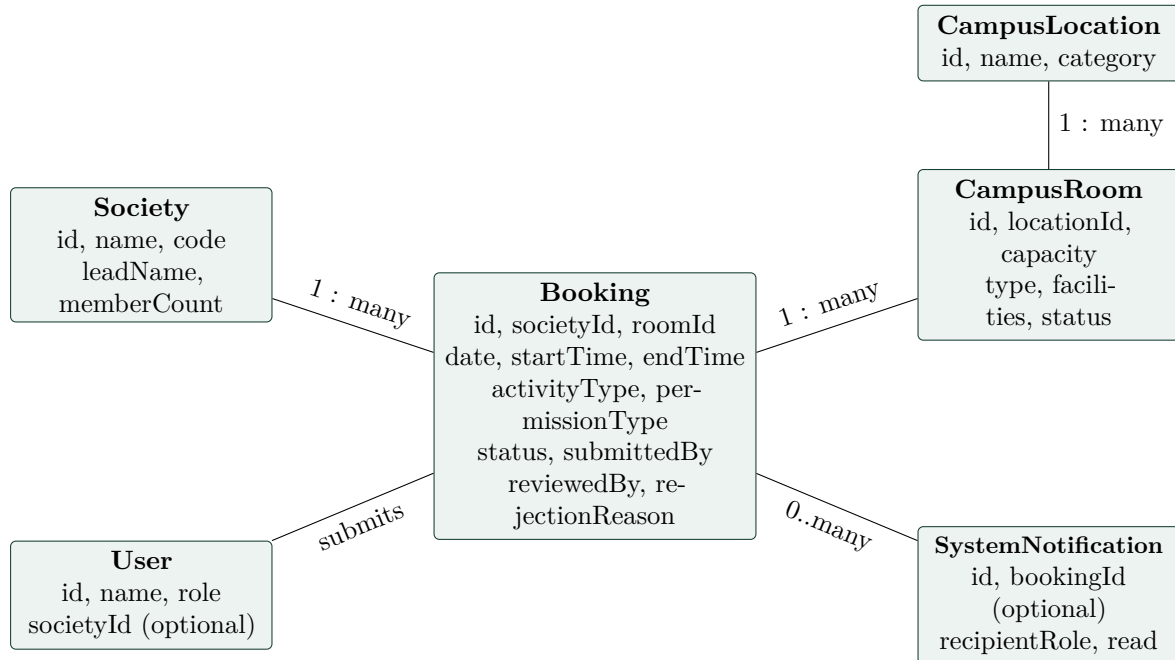
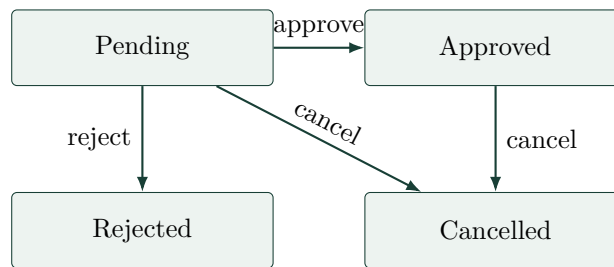


Figure 2.6: Conceptual relationships derived from source interfaces.

2.5.1 Reservation Status Model



This diagram communicates the intended interface lifecycle. The storage service accepts a status value directly, so production software should enforce allowed transitions and permissions on the server.

2.6 Data Flow Diagrams

2.6.1 DFD Level 0: System Context

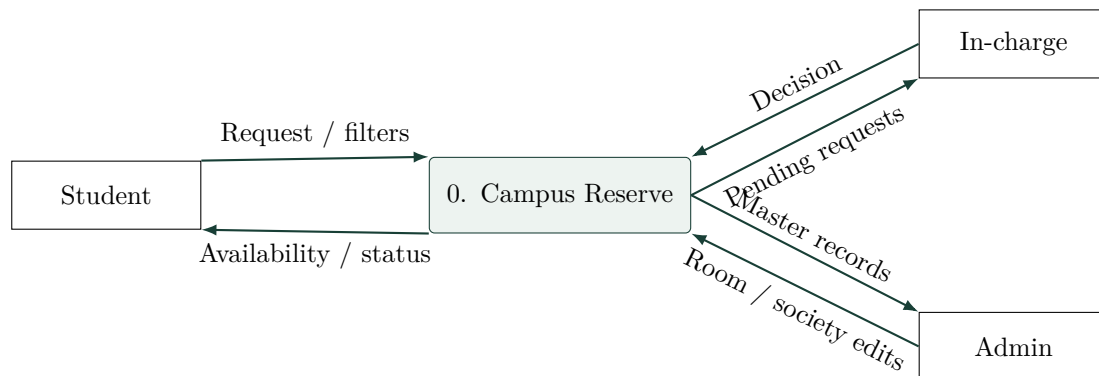


Figure 2.7: External data exchanges with the browser application.

2.6.2 DFD Level 1: Core Processes

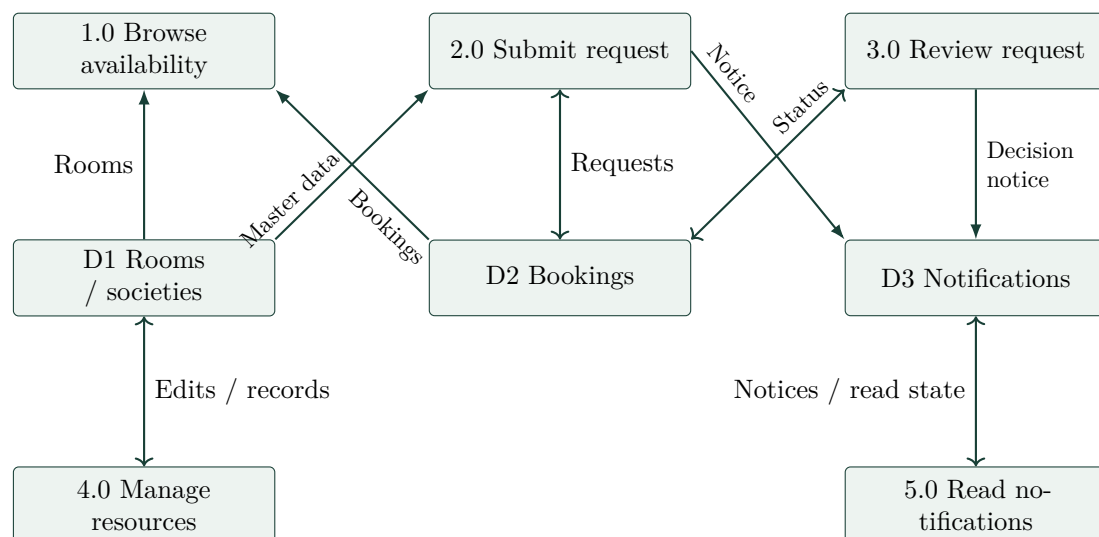


Figure 2.8: Logical decomposition. D1–D3 represent collections in localStorage.

The actor exchanges in Level 0 connect to their respective Level 1 processes: students use 1.0, 2.0 and booking tracking; the in-charge uses 3.0; the administrator uses 4.0. Users read notifications through 5.0. Booking tracking reads D2 and cancellation updates D2 through the storage service.

2.6.3 DFD Level 2: Reservation Validation

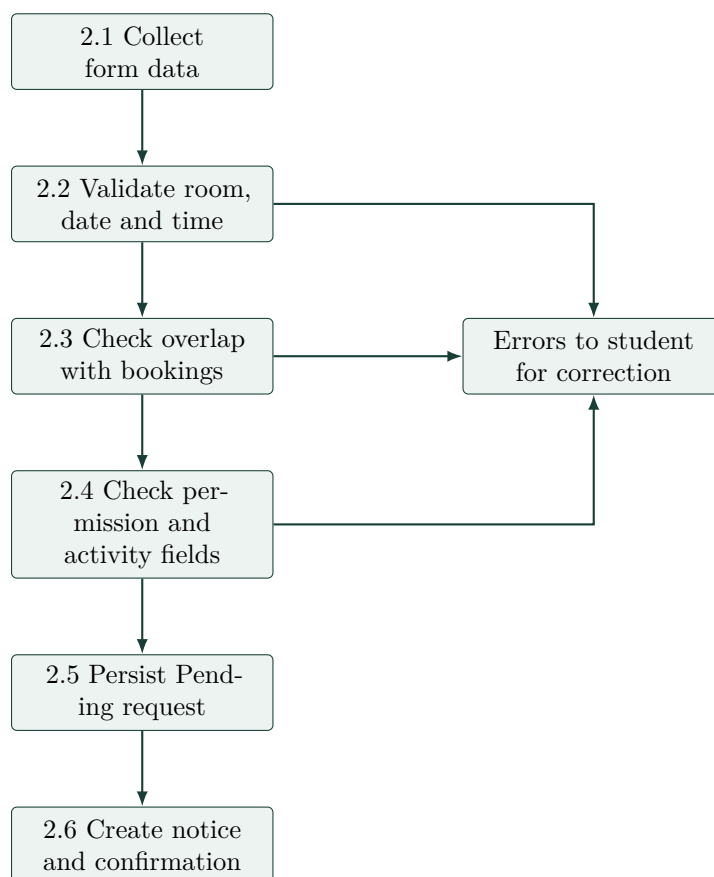


Figure 2.9: Decomposition of request submission. Each failed check prevents progression.

2.6.4 Conflict Rule and Examples

For the same room and date, the service identifies overlap when

$$start_{new} < end_{existing} \quad \wedge \quad end_{new} > start_{existing}.$$

Only Pending and Approved records enter this comparison. The implementation compares normalized time strings in HH:mm format.

Existing booking	New request	Expected comparison
10:00–11:00, Approved	10:30–11:30	Conflict
10:00–11:00, Pending	11:00–12:00	No overlap
10:00–11:00, Rejected	10:30–11:30	Does not block

These are explanatory examples, not recorded executions. A shared deployment needs an atomic server-side conflict check because separate browser stores cannot coordinate simultaneous users.

2.7 Software Requirements Specification

2.7.1 Introduction and Overall Description

Purpose: specify the behaviour of Campus Reserve for an academic prototype and identify requirements for a future shared service. The audience includes the project team, faculty evaluator and future maintainers.

Product perspective: a browser-based single-page application with seeded demo data and local persistence. React Router maps URLs to pages, AuthContext supplies a demo user, and the storage service handles data reads and writes.

User classes: students submit and track requests, in-charges review them, and administrators maintain master records. These roles describe workflows, not secure authorization guarantees.

Operating environment: a modern browser with JavaScript and localStorage. Development uses Node.js and Vite. The repository README specifies Node.js 22.12 or newer. The campus photograph requires internet access and has a typographic fallback.

2.7.2 Functional Requirements

ID	Requirement	Evidence
FR-01	The interface shall offer student, in-charge and admin demo personas.	README
FR-02	The schedule shall filter availability by location, room and date.	Schedule view
FR-03	Selecting an hourly slot shall populate the booking form.	Schedule and form
FR-04	The reservation form shall collect activity, society, location, room, time, permission and activity details before review.	Booking form
FR-05	Validation shall reject unavailable rooms, past dates, reversed intervals and overlapping active reservations.	Form, storage service
FR-06	Requests ending after 20:00 shall require Night Permission.	Booking form
FR-07	Submission shall create a Pending booking and an in-charge notification.	Storage service
FR-08	The booking screen shall support search, status filters, details and cancellation.	README
FR-09	The in-charge shall be able to approve or reject requests.	README, review page
FR-10	The admin shall be able to add or update rooms and societies.	README, storage service
FR-11	Notifications shall support persistent read state.	Storage service

2.7.3 Non-functional Requirements and Constraints

Area	Prototype evidence	Production requirement / target
Usability	Responsive navigation, focus styles and dialog handling are documented.	Validate keyboard and mobile journeys with users.
Security	Passwords are not verified; data is client-controlled.	Verified identity and server-side role/record authorization.
Consistency	Persistence uses one browser's localStorage.	Shared database and atomic conflict prevention.
Reliability	Storage failures can be logged by the service.	Surface failed writes and provide backup/recovery.
Performance	Local filtering and overlap comparisons exist.	Measure response times with representative data and concurrent users.
Maintainability	Typed records and a central storage service separate concerns.	Preserve interfaces while replacing local storage with an API.
Privacy	Notifications route by role in the demo.	Deliver messages only to authorized individual recipients.

No numerical performance target or uptime guarantee has been measured for this report. Such targets should be agreed before a campus pilot.

2.7.4 External Interfaces

User interface: overview, availability schedule, booking wizard, booking list, notification view, in-charge queue and admin management screens.

Software interface: the present application calls a TypeScript storage service and browser APIs. It has no application server API or database connection. The external photograph URL is a display asset, not a booking service.

Data interface: rooms, societies, locations, bookings and notifications serialize as JSON in named localStorage keys. User context provides demo identity information.

2.7.5 Acceptance Boundary

A successful prototype demonstration shows the requested workflow in the same browser. It does not establish secure multi-user deployment, cross-device synchronization or institutional approval authority. Those capabilities require additional implementation and validation.

2.8 User Stories and Story Cards

ID	Story and acceptance criteria	Priority
US-01	As a society representative, I want to inspect room availability so I can choose a suitable slot. Acceptance: changing location, room or date updates the schedule; selecting a slot carries its details to the form.	High
US-02	As a student, I want to submit a complete reservation so the in-charge can review it. Acceptance: valid submission creates a Pending record with a booking ID.	High
US-03	As a student, I want conflict feedback so I can revise an overlapping request. Acceptance: an overlap with Pending or Approved bookings blocks progression; adjacent intervals do not overlap.	High
US-04	As an in-charge, I want to record a decision so students can see the outcome. Acceptance: the record stores Approved or Rejected with available review details, and a notification is created.	High
US-05	As a student, I want to cancel a booking I no longer need. Acceptance: confirming the offered cancellation action stores Cancelled and updates the view.	Medium
US-06	As an administrator, I want to maintain rooms and societies so the prototype uses current master data. Acceptance: a saved add/edit operation appears in the relevant list.	High
US-07	As a user, I want read-state tracking so I can distinguish new notifications. Acceptance: marking a message read persists after refreshing the same browser.	Medium

Priorities are proposed for evaluation planning. No velocity or story-point estimates are asserted because a team estimation session was not supplied.

Chapter 3

Prototype Design and Technology

3.1 Technology Stack

Technology	Declared version	Role
React / React DOM	19.2.8 range	Component-based interface and rendering
TypeScript	6.0.2 range	Typed records and application logic
Vite	8.3.0 range	Development server and production build
React Router DOM	7.18.3 range	Browser routes and navigation
Tailwind CSS	4.3.3 range	Utility CSS and Vite integration
Lucide React	1.44.0 range	Interface icons
Browser localStorage	Browser API	Local JSON persistence
Oxlint	1.81.0 range	Static lint tooling

Versions summarize `package.json` declarations, which use caret or tilde ranges. The lockfile determines exact installed versions. The prototype also uses custom CSS in `src/design/portal.css`. There is no Node.js backend merely because Node.js runs the build tools.

3.2 System Architecture

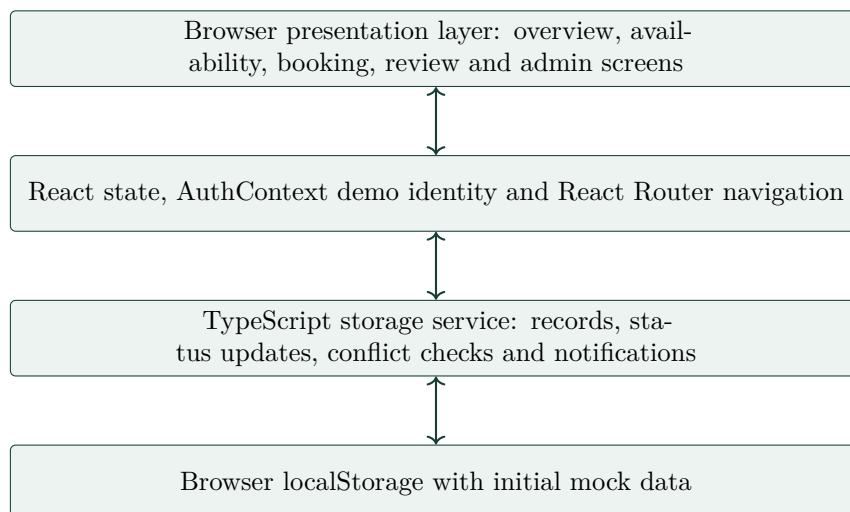


Figure 3.1: Current architecture: all application layers execute on the client.

3.3 Prototype Screens and Navigation

Screen	Purpose	Route / module
Sign-in	Switch demo persona	/login
Campus overview	View society booking totals and recent requests	/student/dashboard
Availability	Inspect hourly room schedule	/availability
Book resource	Complete eight request steps	/student/book
My Bookings	Search, inspect and cancel requests	/student/bookings
Notifications	View messages and read state	/student/notifications
In-charge dashboard	Review pending or all requests	/incharge/dashboard
Admin rooms / societies	Maintain master records	/admin/rooms, /admin/societies

3.4 Eight-step Booking Prototype

1. **Activity:** Society Preparation, Workshop / Session, or Event.
2. **Society:** search and select the organizing society.
3. **Location:** choose the campus building or space category.
4. **Room:** choose the room and avoid records marked Unavailable.
5. **Date and time:** supply a valid interval and check for conflicts.
6. **Permission:** select Morning Permission or Night Permission.
7. **Details:** complete contextual preparation, workshop or event fields.
8. **Review:** check the complete request before submission.

3.5 Illustrative Demonstration Scenario

A society representative plans a workshop for a future date. They select an available room and a 14:00–16:00 slot, complete the workshop name, speaker and purpose, and submit the request. The application creates a Pending record. The demonstrator switches to the in-charge persona in the same browser, reviews the request and approves it. Returning to the student persona shows the updated booking and a notification.

Repeat with an overlapping request to demonstrate the conflict message. A request beginning exactly when the existing booking ends illustrates the adjacent-interval rule. These are proposed demonstration inputs, not captured live results or screenshots.

Chapter 4

Validation and Future Scope

4.1 Acceptance Test Plan

ID	Scenario	Expected result
T01	Switch between demo personas.	Home/navigation reflects the selected role.
T02	Select a free availability slot.	Form receives room, date and time.
T03	Complete a valid daytime request.	Pending booking and in-charge notice appear.
T04	Request overlapping Pending / Approved time.	Conflict validation blocks progression.
T05	Start at the existing booking's end time.	Overlap check permits the adjacent interval.
T06	Choose an Unavailable room.	Form requests a different room.
T07	Use a past date or end time before start.	Validation rejects the input.
T08	End after 20:00 with Morning Permission.	Form requires Night Permission.
T09	Approve or reject a Pending request.	Status, review details and notice update.
T10	Confirm an offered cancellation action.	Booking becomes Cancelled.
T11	Add/edit room or society data.	Updated data appears after a same-browser refresh.
T12	Mark notifications as read.	Read state persists locally.

4.2 Evidence and Unverified Areas

The repository README reports a successful production build and 32 smoke checks. This report records that as a repository claim; the application tests were not independently rerun during document preparation. The listed scenarios above are an acceptance plan, not a pass/fail test log.

Before evaluation, the team should run the repository build and smoke command, then demonstrate the complete workflow in a browser. Responsive layout, keyboard behaviour, storage errors and role transitions need direct observation. Production review must additionally examine authorization, concurrent requests and data recovery.

4.3 Limitations and Future Scope

1. **Identity and authorization:** add institutional sign-in and server-side enforcement of role and record access.
2. **Shared persistence:** replace localStorage with a backend API and database for cross-device data.
3. **Booking consistency:** make conflict checks and writes atomic, and enforce allowed status transitions.
4. **Notification delivery:** target individual recipients and consider email or campus notification integration.
5. **Validation and audit:** enforce participant/capacity policies, retain a decision audit trail, and improve error recovery.
6. **Evaluation:** collect user feedback and measure performance against agreed criteria before deployment.

These items are proposed extensions and must not be presented as existing functionality.

4.4 Conclusion

Campus Reserve demonstrates the core campus booking journey: discover a room, submit a validated request, review the request and track its status. Its separation of screens, typed data and storage operations provides a useful basis for explaining software engineering analysis and prototype design. The current deliverable remains a browser demonstration. A campus-wide service requires secure identity, shared storage and server-enforced booking rules.

References

1. Project repository, accessed 15 September 2026:
https://github.com/rishitajain7/campus_resource_managment.
2. Project overview, prototype limitations and documented verification:
[README.md](#).
3. Declared dependency versions and scripts: [package.json](#).
4. Data records and roles: [TypeScript data interfaces](#).
5. Local persistence, overlap detection and notification creation: [src/services/storageService.ts](#).
6. Screen routing: [src/App.tsx](#).
7. Request steps and validation: [src/pages/BookResourcePage.tsx](#), particularly the form state and validation logic.
8. Supplied format reference: *UCS503_MidSem_Report_LaTeX (2).pdf*. Used for academic organization and cover structure only.

Repository snapshot: the inspected main tree resolved to:

153cd3a888d48b0fce656e6a1b4ae696a1815efe

Links above point to the main branch and may change with later edits.